A Replication and Analysis of the Pixel Stealing Attack

Jeremy Sheng, Joy Chen, Maddie Dreher, Diya Shrawagi
The University of Texas at Austin
{js224768,cc75925,mbd2325,ds54998}@eid.utexas.edu

Abstract

Modern web browsers enforce isolation between websites so that a malicious webpage should not be able to directly read the contents of another origin. However, side-channel attacks can sometimes infer information indirectly through shared browser and hardware resources. This project attempts to replicate the pixel stealing attack proposed by Andrysco et al. [1], which exploits hardware timing differences when computing on subnormal numbers. We first begin by investigating subnormal timing differences on different CPUs to determine the viability of the method and the potential attack surface. We then set up attacker and victim websites, confirm that the same-origin policy prevents direct access of the victim, then attempt to infer pixel colors from timing differences.

1 Introduction

Browsers are designed around the same-origin policy, which prevents one website from directly inspecting the contents of another website. Despite this isolation, webpages still share underlying resources such as the CPU, GPU, caches, and browser rendering system. Differences in how these resources behave while processing particular visual content can potentially leak information through a side channel.

There are various methods to leak pixel information, including exploitation of software timing differences [3], hardware timing differences [1], GPU compression [4], and microarchitectural cache state [2]. Our initial approach is to exploit hardware timing differences of subnormal floating point computation.

2 Background

2.1 IEEE 754 Subnormal Floating-Point Numbers

The IEEE 754 standard for single-precision floating-point arithmetic allocates 32 bits into three fields: 1 sign bit (s), 8 exponent bits (e, with a bias of 127), and 23 significand bits (f). For standard normalized values ($e \in [1, 254]$), the representation assumes an implicit leading bit of 1:

$$x_{\text{norm}} = (-1)^s \times (1.f)_2 \times 2^{e-127} \tag{1}$$

The smallest positive normalized single-precision number occurs when $e = 1$ and $f = 0$, yielding $N_{\min} = 1.0_2 \times 2^{-126} \approx 1.175 \times 10^{-38}$.

When the exponent field is all zeros ($e = 0$), the value transitions to a subnormal (or denormal) representation. In subnormal mode, the implicit leading bit shifts from 1 to 0, and the effective exponent is fixed at -126:

$$x_{\text{sub}} = (-1)^s \times (0.f)_2 \times 2^{-126} \tag{2}$$

The smallest representable positive subnormal number has a significand bit pattern of `0x00000001`, corresponding to $2^{-149} \approx 1.4 \times 10^{-45}$. Subnormals support gradual underflow, bridging the gap between 0.0 and $N_{\min}$. This gradual decay preserves fundamental mathematical identities, guaranteeing that subtracting two distinct floating-point numbers will not evaluate to zero ($x \neq y \implies x - y \neq 0$), thereby preventing unexpected division-by-zero exceptions in numerical algorithms.

2.2 Microarchitectural Timing Asynchrony

Modern x86 execution units are engineered primarily for normalized floating-point operations, executing scalar or vector arithmetic in 1 to 4 clock cycles ($\sim$ 1 ns). Implementing full dedicated hardware logic for subnormal normalization across all execution units requires substantial silicon area and power consumption.

Consequently, when an execution pipeline encounters a subnormal operand or generates a subnormal output under standard IEEE 754 compliance (where Flush-to-Zero and Denormals-are-Zero modes are disabled), the fast execution datapath cannot process the value. The CPU flushes the out-of-order execution pipeline and delegates processing to internal microcode assist routines (`fp_assists`). Handling gradual underflow via microcode introduces a severe latency penalty of 80 to 200+ cycles ($\sim$ 30–70 ns) per operation, creating a data-dependent timing side-channel.

2.3 Pixel Stealing Attack

Web browsers enforce the Same-Origin Policy (SOP) to prevent malicious sites from accessing private cross-origin content. Although browsers allow cross-origin images to be embedded visually, direct pixel inspection via APIs such as HTML5 canvas element extraction is explicitly blocked. Andrysco et al. [1] demonstrated that subnormal execution timing can be exploited to bypass these SOP restrictions and reconstruct cross-origin pixel data over the web:

1. **SVG Filter Processing:** A target cross-origin image is rendered inside an HTML5/SVG container. The attacker applies a custom SVG filter primitive (such as `feColorMatrix` or `feConvolveMatrix`) that scales normalized pixel color channel values $I \in [0.0, 1.0]$ by a minute scalar $\alpha \approx 10^{-42}$.
2. **Data-Dependent Latency:**

- Non-Black Pixels ($I > 0$): Multiplying a non-zero color intensity by α yields a subnormal result ($\alpha I \approx 10^{-42}$). This repeatedly triggers microcode assist traps during rendering, significantly stalling the browser execution pipeline.
- Black Pixels ($I = 0$): Multiplying zero by α yields exact zero (0.0), completing immediately on the fast hardware execution path without latency.

3. **Pixel Extraction:** By measuring frame render latencies using browser timing mechanisms (such as requestAnimationFrame or CSS repaint callbacks), the attacker classifies individual pixels as black or non-black based on rendering speed. Repeating this process across a target pixel grid progressively leaks private visual content.

3 Threat Model

We assume that an attacker controls a malicious website and convinces a user to visit it. The attacker does not have direct access to the victim website's DOM, memory, network traffic, or authentication information. Instead, the attacker can only execute ordinary browser code from the malicious origin and observe side-channel measurements exposed through shared browser or hardware behavior.

For our experiments, the victim is a webpage that we control and populate with a small 8x8 grid of pixels, half of which are black, and half of which are white. This gives us ground truth for evaluating the attack.

4 Our Approach

Since our intended method of leaking pixel information is to exploit subnormal timing differences as done in [1], we began by first gathering data on various CPUs. We developed two programs to evaluate the performance of floating-point addition and multiplication using both normal and subnormal operands. Separate implementations were written for the ARM and x86 ISAs, allowing us to measure execution times across a range of processors. By comparing the timing differences between normal and subnormal operations, we investigated how these variations depend on the underlying ISA and processor architecture, identifying potential targets for our subsequent attack. Preliminary results demonstrated that the method of exploiting subnormal timing differences is viable for our attack, at least on some processors. Results are discussed in further detail in Section 5.

TODO TODO TODO TODO TODO We also began setting up the attack. So far, we have set up a victim website, consisting of an 8x8 grid of black and white pixels, and an attack website, which is able to extract a pixel and measure timing, but does not apply any SVG filters, and as such does not exhibit any timing differences. This is left to future work.

5 Evaluation

5.1 Timing Data

We collected timing data for floating-point adds and multiplies across a wide range of ARM and x86 processors, which were sourced from either personal laptops or server compute machines, such as the ECE LRC machines. The results are summarized in Figure 1.

On the Apple M1 and M5 chips, there is no significant timing difference between normal and subnormal operands for both add and multiply. This is expected, since Apple silicon has full hardware support for subnormal floating-point operations and can execute at full speed.

On the four Intel chips, including the Intel Xeon 5218 used by the ECE LRC machines, we see significant deltas between normal and subnormal execution. This is due to the microcode assist sequence discussed in Section 2. The pipeline must be flushed and a special microcode sequence must occur to correctly handle subnormal execution, which is very slow.

TODO: what is happening on AMD EPYC Milan?

TODO: what is happening on NVIDIA Grace?

This data provides valuable information for the direction of our attack. Apple chips, AMD EPYC Milan, and NVIDIA Grace are out of the question if we continue with the method of exploiting subnormal timing differences, since they have no measurable difference. We will likely proceed using the Intel Xeon 5218, since we have easy access to it via the ECE LRC servers, and it exhibits dramatic timing differences.

5.2 The Attack Setup

TODO: detail the current status of our attack setup, and any next steps

6 Related Work

Stone's *Pixel Perfect Timing Attacks with HTML5* [3] demonstrated the feasibility of cross-origin pixel stealing through browser rendering behavior. By exploiting software timing differences in the HTML5 `requestAnimationFrame` API, the attack measures how quickly CSS and SVG filters process pixels within a cross-origin iframe, allowing an attacker to infer image contents. This work established browser rendering operations as an unintended information channel and laid the foundation for subsequent SVG-based timing attacks.

Andrysco et al. [1] extended this approach by identifying hardware-level timing differences in floating-point operations. They demonstrated that arithmetic involving subnormal floating-point numbers can execute significantly slower than operations on normal numbers. By constructing SVG filters that trigger these operations, attackers can distinguish pixel values based on execution time. Unlike Stone's approach, which exploits browser rendering behavior, this

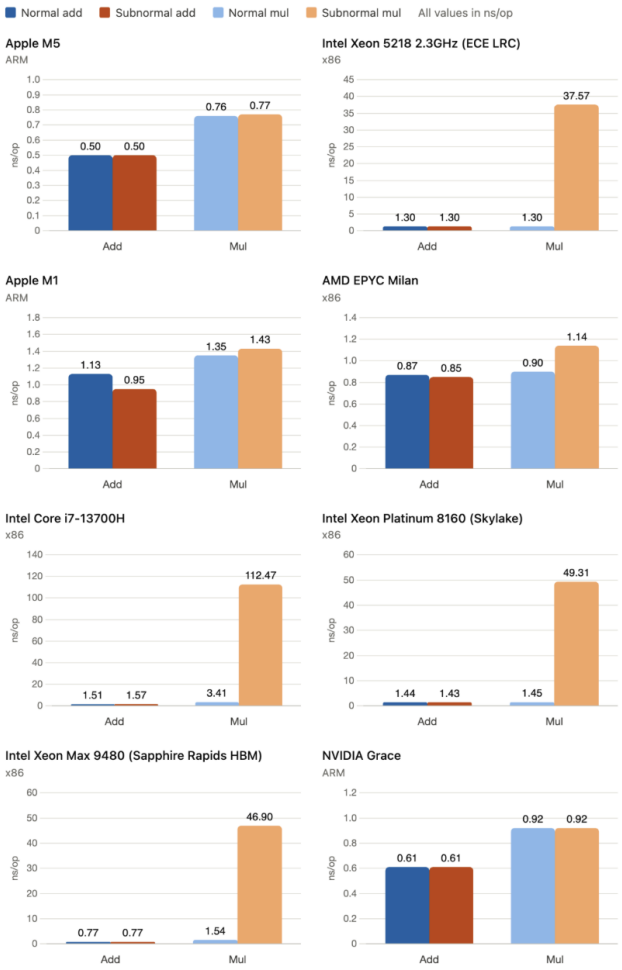


Figure 1. Time to perform floating-point instructions, measured in ns/op. Note the y-axis for each graph is different.

attack leverages CPU-level floating-point performance characteristics, demonstrating how hardware behavior can undermine seemingly constant-time software.

Wang et al.'s *GPU.zip* [4] expanded side-channel research to integrated GPU architectures by exploiting hardware-based graphical data compression. The authors demonstrated that undocumented, lossless compression mechanisms can introduce data-dependent differences in DRAM traffic and rendering time. By constructing carefully designed SVG filters that produce highly compressible pixel patterns, attackers can infer underlying pixel values. This work showed that hardware optimizations intended to improve performance can themselves introduce information leakage, extending earlier attacks beyond CPU execution timing.

O'Connell et al.'s *Pixel Thief* [2] further advanced pixel-stealing attacks by exploiting CPU cache-state changes rather than relying directly on elapsed execution time. By monitoring the cache footprint left by SVG filter execution, the attack extracts pixel information despite modern browser timing mitigations, including reduced timer resolution and jitter. Building on the SVG filter leakage demonstrated by earlier work, Pixel Thief introduces an alternative measurement technique based on microarchitectural cache behavior, achieving extraction rates of up to 267 bits per second.

Collectively, these works demonstrate the evolution of browser-based side-channel attacks from software-level rendering measurements to hardware-level information leakage. Stone established the feasibility of timing-based pixel extraction, Andrysco et al. identified CPU floating-point timing differences, GPU.zip exposed leakage through GPU memory compression, and Pixel Thief demonstrated cache-based extraction. Together, they highlight the limitations of software-level timing mitigations and the broader security implications of data-dependent hardware behavior.

7 Conclusion

This project is awesome.

References

[1] Andrysco, M., Kohlbrenner, D., Mowery, K., Jhala, R., Lerner, S., and Shacham, H. On subnormal floating point and abnormal timing. In *2015 IEEE Symposium on Security and Privacy* (May 2015), IEEE.

[2] O'Connell, S., Sour, L. A., Magen, R., Genkin, D., Oren, Y., Shacham, H., and Yarom, Y. Pixel thief: Exploiting SVG filter leakage in firefox and chrome. In *33rd USENIX Security Symposium (USENIX Security 24)* (2024), USENIX Association, pp. 3331–3348.

[3] Stone, P. Pixel perfect timing attacks with html5. Black Hat USA, July 2013.

[4] Wang, Y., Paccagnella, R., Gang, Z., Vasquez, W., Kohlbrenner, D., Shacham, H., and Fletcher, C. GPU.zip: On the side-channel implications of hardware-based graphical data compression. In *2024 IEEE Symposium on Security and Privacy (SP)* (2024), IEEE.